Practical Week-9

1.Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

vedadripinninti@MSI123

GNU nano 8.7.1 src/file_copy_syscalls.c *

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int source, destination;
    char buffer[1024];
    ssize_t bytes;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");

    return 0;
}
```

Output:

vedadripinninti@MSI123

```
vedadripinninti@MSI123:~$ cd OS_Skill
mkdir -p Week-9/src Week-9/bin
cd Week-9
vedadripinninti@MSI123:~/OS_Skill/Week-9$ nano src/file_copy_syscalls.c

vedadripinninti@MSI123:~/OS_Skill/Week-9$ ./bin/file_copy_syscalls input.txt copy.txt
-bash: ./bin/file_copy_syscalls: No such file or directory
vedadripinninti@MSI123:~/OS_Skill/Week-9$ echo "Hello vedadripinninti! This is a file copy test." > input.txt
vedadripinninti@MSI123:~/OS_Skill/Week-9$ cat input.txt
Hello vedadripinninti! This is a file copy test.
vedadripinninti@MSI123:~/OS_Skill/Week-9$ gcc src/file_copy_syscalls.c -o bin/file_copy_syscalls
vedadripinninti@MSI123:~/OS_Skill/Week-9$ ./bin/file_copy_syscalls input.txt copy.txt
File copied successfully using system calls.
vedadripinninti@MSI123:~/OS_Skill/Week-9$ cat copy.txt
Hello vedadripinninti! This is a file copy test.
```

Modified program to use lseek():

vedadripinninti@MSI123

GNU nano 8.7.1 src/file_copy_syscalls.c *

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
```

```c
    int source, destination;
    char buffer[1024];
    ssize_t bytes;
    off_t size;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    size = lseek(source, 0, SEEK_END);
    printf("Source file size: %ld bytes\n", (long)size);
    lseek(source, 0, SEEK_SET);

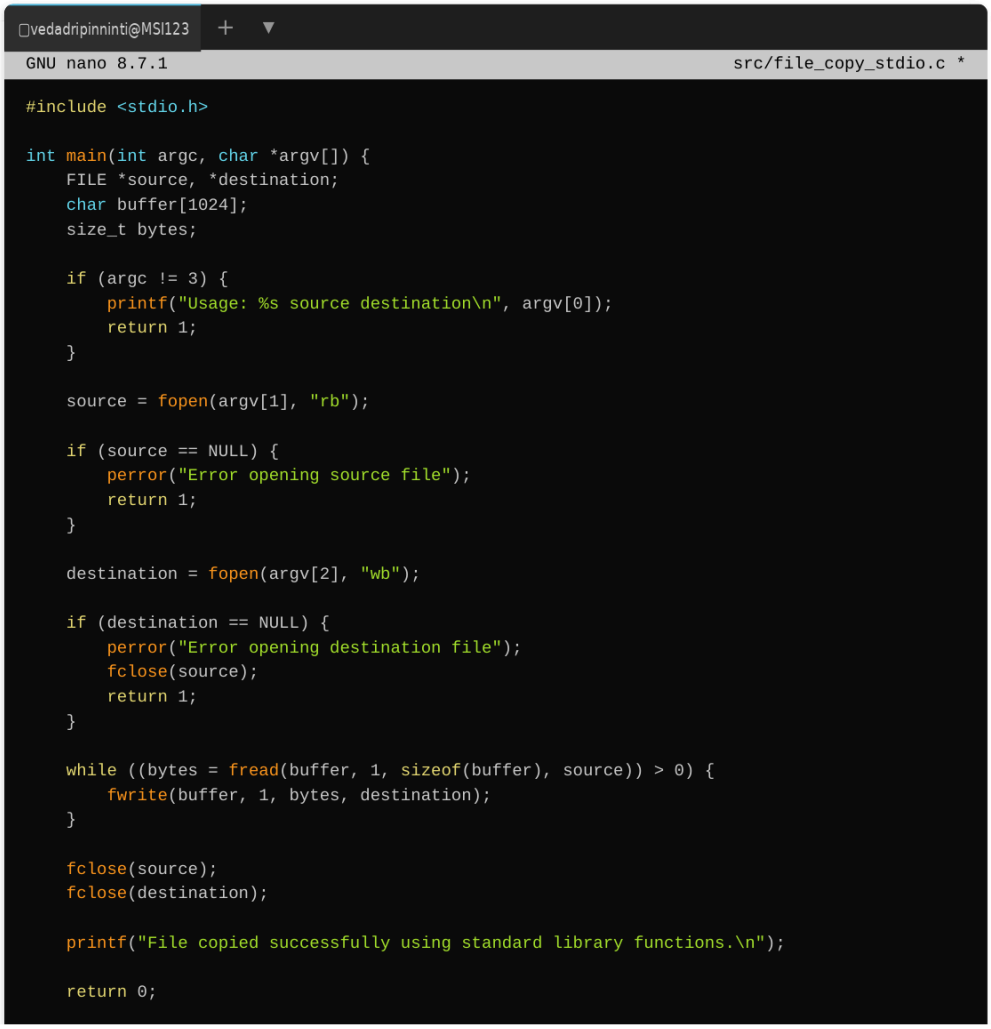
    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");

    return 0;
}
```

1B




```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    FILE *source, *destination;
    char buffer[1024];
    size_t bytes;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");

    if (source == NULL) {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");

    if (destination == NULL) {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytes = fread(buffer, 1, sizeof(buffer), source)) > 0) {
        fwrite(buffer, 1, bytes, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
```

```
}
```

Output:

```
vedadripinninti@MSI123
vedadripinninti@MSI:~/Skill/Week-9$ nano src/file_copy_stdio.c
vedadripinninti@MSI:~/Skill/Week-9$ gcc src/file_copy_stdio.c -o bin/file_copy_stdio
vedadripinninti@MSI:~/Skill/Week-9$ ./bin/file_copy_stdio input.txt copy2.txt
File copied successfully using standard library functions.
vedadripinninti@MSI:~/Skill/Week-9$ cat copy2.txt
Hello vedadripinninti! This is a file copy test.
```

2. Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
vedadripinninti@MSI123
GNU nano 8.7.1                                    src/io_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input_fd, output_fd;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0) {
        perror("input.txt");
        return 1;
    }

    output_fd = open("output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd < 0) {
        perror("output.txt");
        close(input_fd);
        return 1;
    }

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    char buffer[100];

    printf("This output is redirected to output.txt\n");

    printf("Reading from redirected input:\n");

    while (read(STDIN_FILENO, buffer, sizeof(buffer) - 1) > 0) {
        break;
    }

    return 0;
}
```

Output:

```
vedadripinninti@MSI123
vedadripinninti@MSI:~/Skill/Week-9$ gcc src/io_redirection.c -o bin/io_redirection
vedadripinninti@MSI:~/Skill/Week-9$ ./bin/io_redirection
vedadripinninti@MSI:~/Skill/Week-9$ cat output.txt
This output is redirected to output.txt
Reading from redirected input:
```